TÀI LIỆU ĐẶC TẢ THIẾT KẾ HỆ THỐNG

HỆ THỐNG HỎI ĐÁP VÀ TRA CỨU VĂN BẢN QUY PHẠM PHÁP LUẬT VIỆT NAM

Thông tin Quản lý Thiết kế	Chi tiết Kỹ thuật
Tên Hệ thống	Hệ thống Hỏi đáp Văn bản Quy phạm Pháp luật
Mã Tài liệu	SDD-LEGAL-RAG-001
Tài liệu Liên kết	SRS-LEGAL-RAG-001 Phiên bản 2.0
Phiên bản	3.0
Ngày Ban hành	18/10/2026
Tiêu chuẩn Áp dụng	IEEE 1016-2009

CHƯƠNG 1. KIÊN TRÚC HỆ THỐNG TÔNG THÊ

1.1. Mục tiêu Thiết kế và Nguyên tắc Kiến trúc

Tài liệu này mô tả thiết kế kỹ thuật chi tiết theo chuẩn IEEE 1016-2009 để hiện thực hóa các yêu cầu trong SRS-LEGAL-RAG-001. Hệ thống được thiết kế theo mô hình kiến trúc phân tán kết hợp giữa máy trạm cục bộ và hạ tầng đám mây miễn phí nhằm tối ưu tài nguyên phần cứng:

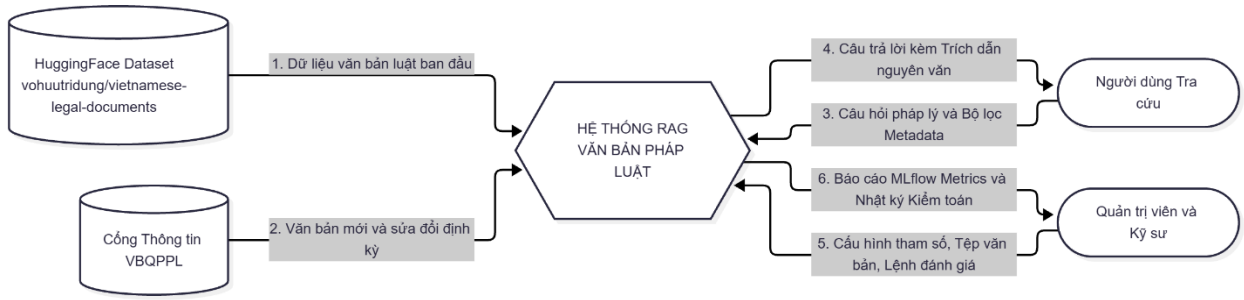
1.2. Bảng Đặc tả Công nghệ Thực thi

Hệ thống chốt cố định một bộ công nghệ duy nhất để triển khai xuyên suốt:

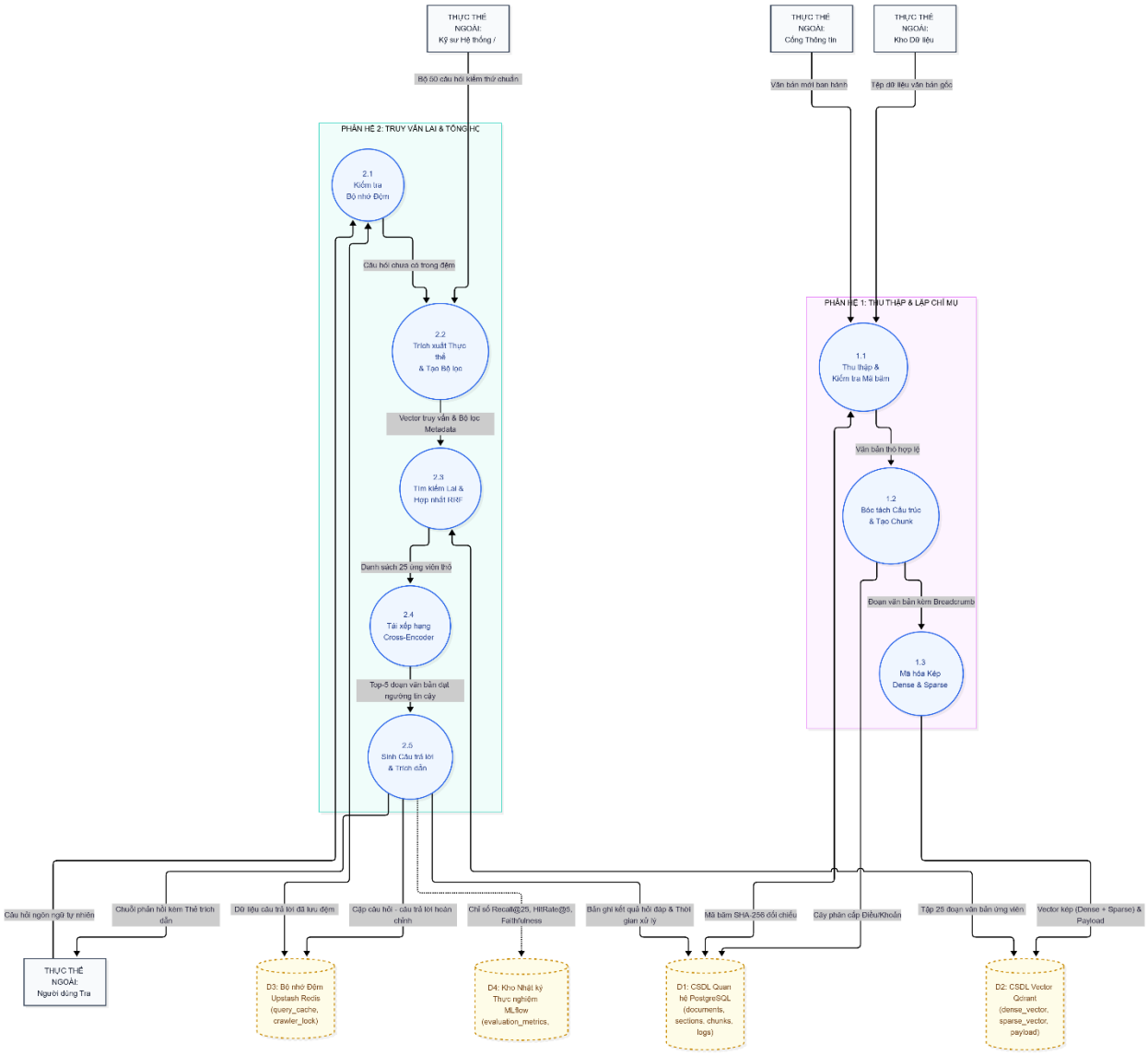
Tầng Kiến trúc	Công nghệ Sử dụng	Môi trường Chạy	Vai trò Kỹ thuật Cụ thể
Nguồn Dữ liệu Nền	HuggingFace datasets	Google Colab / Máy cục bộ	Tải luông dữ liệu từ kho vohuutridung/vietnamese-legal-documents.
Bộ Cào Định kỳ	Playwright, BeautifulSoup4	GitHub Actions Cron	Quét văn bản mới ban hành hàng tuần và kiểm tra mã băm SHA-256.
Bộ Phân tích Văn bản	IBM Docling, PyMuPDF, Regex	Google Colab / Máy cục bộ	Bóc tách văn bản theo phân cấp Chương, Mục, Điều, Khoản và giữ bảng Markdown.
Mô hình Embedding	BAAI/bge-m3	Google Colab GPU / ONNX CPU	Sinh đồng thời Dense Vector 1024 chiều và Sparse Lexical Weights.
Mô hình Reranker	BAAI/bge-reranker-v2-m3	ONNX Runtime trên CPU	Chấm điểm tương quan trực tiếp giữa câu hỏi và 25 ứng viên để lọc lấy Top-5.
Cơ sở Dữ liệu Vector	Qdrant Cloud	Cloud Free-Tier	Lưu trữ vector kép, đánh chỉ mục HNSW và Inverted Index, hỗ trợ lọc Metadata.
Cơ sở Dữ liệu Quan hệ	Neon PostgreSQL 16	Cloud Free-Tier	Lưu danh mục văn bản gốc, trạng thái hiệu lực, lịch sử cào và nhật ký kiểm toán.
Bộ đệm và Khóa	Upstash Redis 7	Cloud Free-Tier	Lưu bộ đệm kết quả truy vấn và tạo khóa phân tán chống chạy trùng bộ cào.
Mô hình Ngôn ngữ Lớn	Qwen2.5-7B-Instruct Q4_K_M	Máy trạm Cục bộ	Sinh câu trả lời pháp lý có trích dẫn dựa trên ngữ cảnh được cung cấp.

Tầng Kiến trúc	Công nghệ Sử dụng	Môi trường Chạy	Vai trò Kỹ thuật Cụ thể
Engine Phục vụ LLM	Ollama Server	Máy trạm Cục bộ	Phục vụ mô hình ngôn ngữ lớn qua cổng 11434, chia sẻ tải giữa GPU 4GB và RAM.
Điều phối và Backend	FastAPI, LangGraph, langchain-core	Máy trạm Cục bộ	Điều phôi đô thị trạng thái RAG, kiểm tra ngưỡng tin cậy và phát luồng SSE.
Giao diện Người dùng	Chainlit	Máy trạm Cục bộ	Hiên thị khung hội thoại thời gian thực, bộ lọc pháp lý và thẻ đối chiếu trích dẫn.
Đánh giá Chất lượng	MLflow	Máy trạm Cục bộ	Ghi nhận kêt quả đo lường Recall@25, Hit Rate@5, MRR@5 và Faithfulness.

1.3. Khung nhìn Ngữ cảnh và Luồng Dữ liệu Mức 0



1.4. Khung nhìn Phân rã Luồng Dữ liệu Mức 1

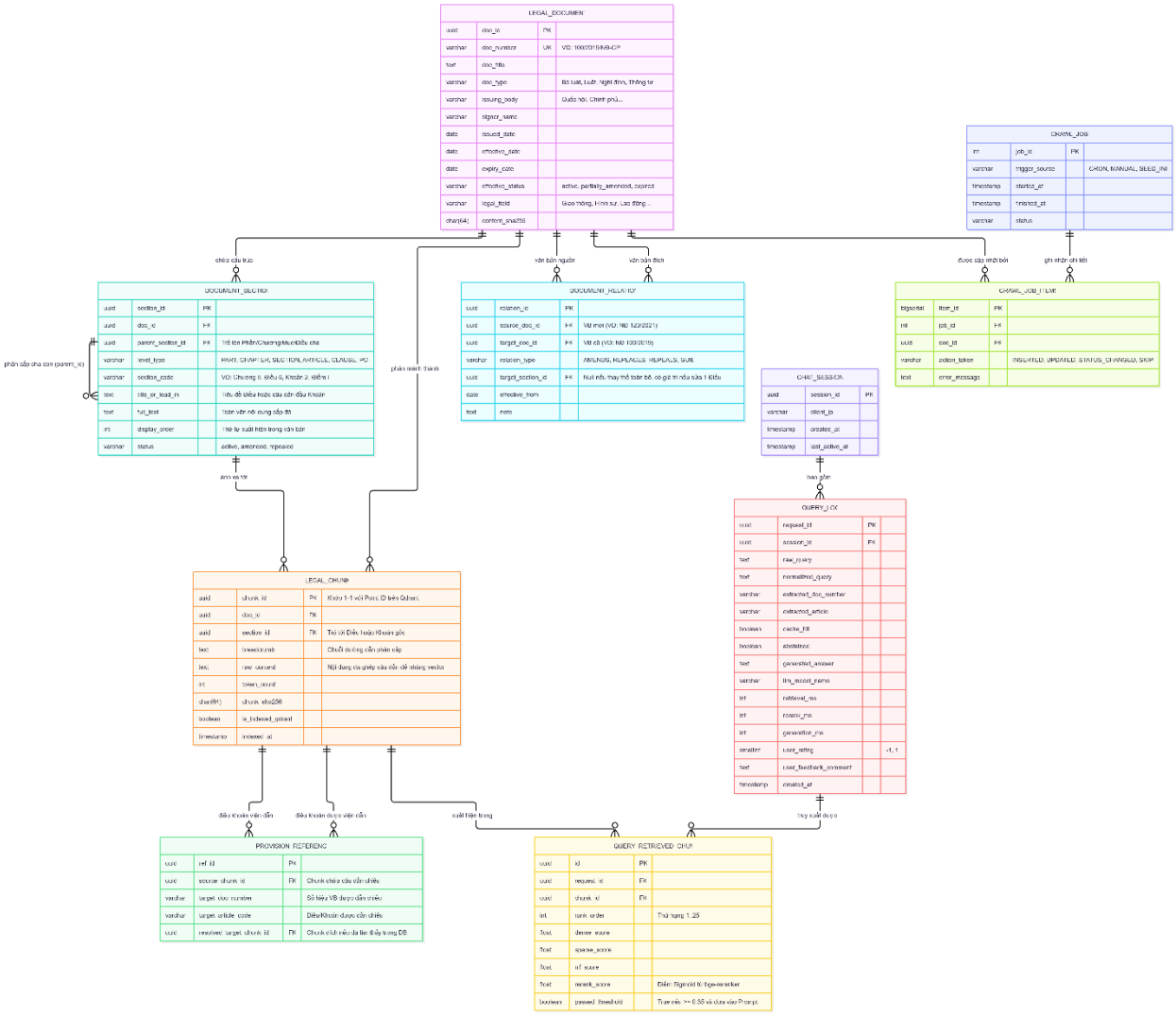


CHƯƠNG 2. THIẾT KẾ DỮ LIỆU VÀ LƯU TRỮ

2.1. Chiến lược Phân tầng Lưu trữ

- Tầng Quan hệ trên Neon PostgreSQL lưu danh mục toàn bộ văn bản quy phạm pháp luật, giữ vai trò đối chiếu mã băm SHA-256 và quản lý trạng thái hiệu lực.
- Tầng Vector trên Qdrant Cloud chỉ lưu trữ các chunk thuộc nhóm Bộ luật, Luật, Nghị định, Thông tư có trạng thái hiệu lực là `active` hoặc `amended`, kết hợp bật lượng tử hóa `INT8 Scalar Quantization` để giữ dung lượng chỉ mục dưới 4GB Disk và 1GB RAM.

2.2. Entity Relation Diagram



2.3. Thiết kế Collection và Payload trên Qdrant Cloud

Collection `legal_chunks_v1` trên Qdrant được thiết kế với cấu hình Named Vectors kết hợp Payload Indexing:from qdrant_client import QdrantClient, models

Tên Vector	Kiểu Biểu diễn	Số chiều	Độ đo Khoảng cách	Thuật toán Chỉ mục & Tham số	Tối ưu Bộ nhớ
dense_vector	Dense Float Vector	1024	Cosine Similarity	HNSW ($M = 16$, $ef_construct = 200$)	Scalar Quantization INT8

sparse_vector	Sparse Lexical Vector	Biến thiên	Inner Product	Inverted Index	Không áp dụng

Tên Trường Payload	**Kiểu Dữ liệu**	**Chỉ mục Lọc (Payload Index)**	**Mục đích Sử dụng**
doc_id	UUID	UUID Index	Khóa ngoại liên kết bảng legal_documents trong PostgreSQL.
doc_number	String	Keyword Index	Lọc chính xác theo số hiệu văn bản (Ví dụ: 100/2019/NĐ-CP).
doc_type	String	Keyword Index	Lọc theo loại văn bản (Bộ luật, Luật, Nghị định, Thông tư).
issued_year	Integer	Integer Index	Lọc hoặc phân vùng văn bản theo năm ban hành.
effective_status	String	Keyword Index	Lọc trạng thái hiệu lực (active, partially_amended, expired).
article_code	String	Keyword Index	Lọc chính xác theo số Điều (Ví dụ: Điều 6).
clause_code	String	Keyword Index	Lọc theo số Khoản (Ví dụ: Khoản 2 hoặc ALL).
breadcrumb	String	Không đánh chỉ mục	Chuỗi phân cấp nguồn gốc dùng hiển thị ngữ cảnh nhanh.
chunk_sha256	String	Keyword Index	Kiểm tra tính bất biến và chống trùng lặp khi nạp dữ liệu.

2.4. Thiết kế Không gian Khóa trên Redis

Tiền tố Khóa (Key Pattern)	**Cấu trúc Định danh**	**Kiểu Dữ liệu Redis**	**Thời gian Sống (TTL)**	**Mục đích Kỹ thuật**
rag:cache:query:{query_hash}	Mã băm SHA-256 của câu hỏi đã chuẩn hóa kèm bộ lọc	String (JSON)	86.400 giây (24h)	Lưu kết quả trả lời và trích dẫn để phản hồi ngay khi trúng bộ đệm.

rag:lock:{job_name}	Tên tiến trình nền (Ví dụ: periodic_crawler)	String	3.600 giây (1h)	Khóa phân tán đảm bảo chỉ có duy nhất 1 tiến trình cào chạy tại một thời điểm.

CHƯƠNG 3. THIẾT KẾ CHI TIẾT CÁC PHÂN HỆ

3.1. Phân hệ Bóc tách Cấu trúc Phân cấp và Sinh Chunk (`parser_service.py`)

3.1.1. Cấu trúc Dữ liệu Đầu ra của mỗi Chunk

```
class LegalChunkRecord(BaseModel):
    chunk_id: str = Field(..., description="Mã UUID5 sinh từ doc_number + article_id + clause_id")
    doc_id: str
    doc_number: str
    doc_title: str
    doc_type: str
    issuing_body: Optional[str] = None
    issued_year: int
    effective_status: str = "active"
    chapter_id: Optional[str] = None
    section_id: Optional[str] = None
    article_id: str
    clause_id: str = "ALL"
    referenced_provisions: List[str] = []
    raw_content: str
```

3.1.2. Thuật toán State Machine Cắt Chunk và Bảo toàn Câu dẫn

1. Đọc văn bản đầu vào từng dòng sau khi đã chuẩn hóa bảng mã Unicode NFC và chuyển đổi bảng biểu sang Markdown.
2. Duy trì bộ nhớ trạng thái hiện hành gồm: `current_chapter`, `current_section`, `current_article_id`, `current_article_title`, `article_lead_in` và danh sách `clauses`.
3. Khi gặp dòng khớp biểu thức `^Điều\s+(\d+[a-zđ]?)\.\s*(.*)$`:

- Đóng gói `Điều` trước đó thành các đối tượng `LegalChunkRecord`.
- Khởi tạo trạng thái cho `Điều` mới; các dòng văn bản xuất hiện trước `Khoản 1` được tích lũy vào biến `article_lead_in`.

4. Quy tắc đóng gói một `Điều`:
   - Đếm tổng số token của toàn bộ `Điều` bằng bộ đếm token của `BAAI/bge-m3`.
   - Nếu tổng số token không vượt quá 1024: Tạo một bản ghi duy nhất với `clause_id = "ALL"`.
   - Nếu tổng số token vượt quá 1024: Tách mỗi `Khoản` thành một bản ghi riêng với `clause_id = "Khoản X"`, trong đó phần nội dung của mỗi bản ghi được ghép theo công thức:

`Breadcrumb + "\n" + Tiêu đề Điều + "\n" + article_lead_in + "\n" + Nội dung Khoản X`.

3.2. Phân hệ Mã hóa Kép và Đồng bộ Vector (`embedding_service.py`)

- Sử dụng mô hình `BAAI/bge-m3` với tham số `return_dense=True, return_sparse=True, max_length=1024`.
- Đối với vector thưa: Chuyển đổi từ điển trọng số từ vựng `{token_id: weight}` của `bge-m3` sang định dạng `models.SparseVector(indices=list_of_token_ids, values=list_of_weights)` tương thích trực tiếp với Qdrant.
- Khóa chính `chunk_id` trong Qdrant được sinh bằng `uuid.uuid5(uuid.NAMESPACE_OID, f"{doc_number}_{article_id}_{clause_id}")` để đảm bảo tính bất biến khi thực thi lệnh `upsert`.

3.3. Phân hệ Truy xuất Lai và Tái xếp hạng (`retriever_service.py`)

3.3.1. Trích xuất Bộ lọc Tiền truy vấn

Trước khi mã hóa câu hỏi, hàm `extract_query_metadata(query: str)` quét câu hỏi bằng regex:

* Nếu phát hiện số hiệu văn bản khớp mẫu `\b(\d+/\d{4}/[A-ZĐ0-9\-]+)\b`: Thêm điều kiện lọc `FieldCondition(key="doc_number", match=MatchValue(value=extracted_doc_num))`.

* Nếu phát hiện số Điều khớp mẫu `\bĐiều\s+(\d+[a-zđ]?)\b`: Thêm điều kiện lọc `FieldCondition(key="article_id", match=MatchValue(value=f"Điều {extracted_art}"))`.

* Luôn đính kèm điều kiện lọc mặc định: `must_not=[FieldCondition(key="effective_status", match=MatchValue(value="expired"))]`.

3.3.2. Thực thi Truy xuất Lai RRF trên Qdrant và Tái xếp hạng Cross-Encoder

Quá trình truy xuất được thực hiện qua hai bước liên tiếp

1. **Truy xuất lai và hợp nhất thứ hạng (Hybrid Search & RRF):**
   - Truy vấn đồng thời hai nhánh trên cơ sở dữ liệu vector cùng bộ lọc siêu dữ liệu: nhánh Dense lấy 25 ứng viên theo khoảng cách Cosine, nhánh Sparse lấy 25 ứng viên theo tích vô hướng.
   - Hợp nhất danh sách ứng viên bằng công thức Reciprocal Rank Fusion với hằng số $k = 60$:

$$Score_{RRF}(d) = \sum_{m \in \{dense, sparse\}} \frac{1}{60 + r_m(d)}$$

   - Giữ lại 25 đoạn trích duy nhất có điểm $Score_{RRF}$ cao nhất[cite: 1, 5].
2. **Tái xếp hạng ngữ nghĩa (Cross-Encoder Reranking):**
   - Ghép 25 ứng viên với câu hỏi gốc thành các cặp văn bản đưa vào mô hình BAAI/bge-reranker-v2-m3.
   - Điểm tương quan được chuẩn hóa qua hàm Sigmoid về đoạn $[0,1]$:

$$Score_{rerank} = \frac{1}{1 + e^{-s}}$$

   - Sắp xếp giảm dần theo $Score_{rerank}$ và lấy 5 đoạn trích có điểm số cao nhất chuyển tiếp sang phân hệ điều phối.

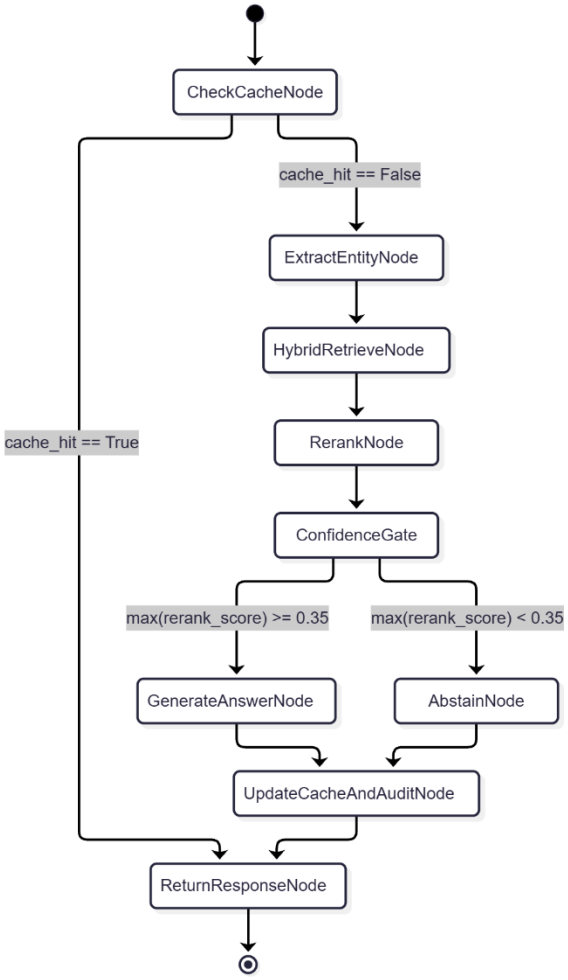
3.4. Phân hệ Điều phối Đồ thị Trạng thái (`orchestrator_service.py`)

3.4.1. Cấu trúc Trạng thái Đồ thị LangGraph

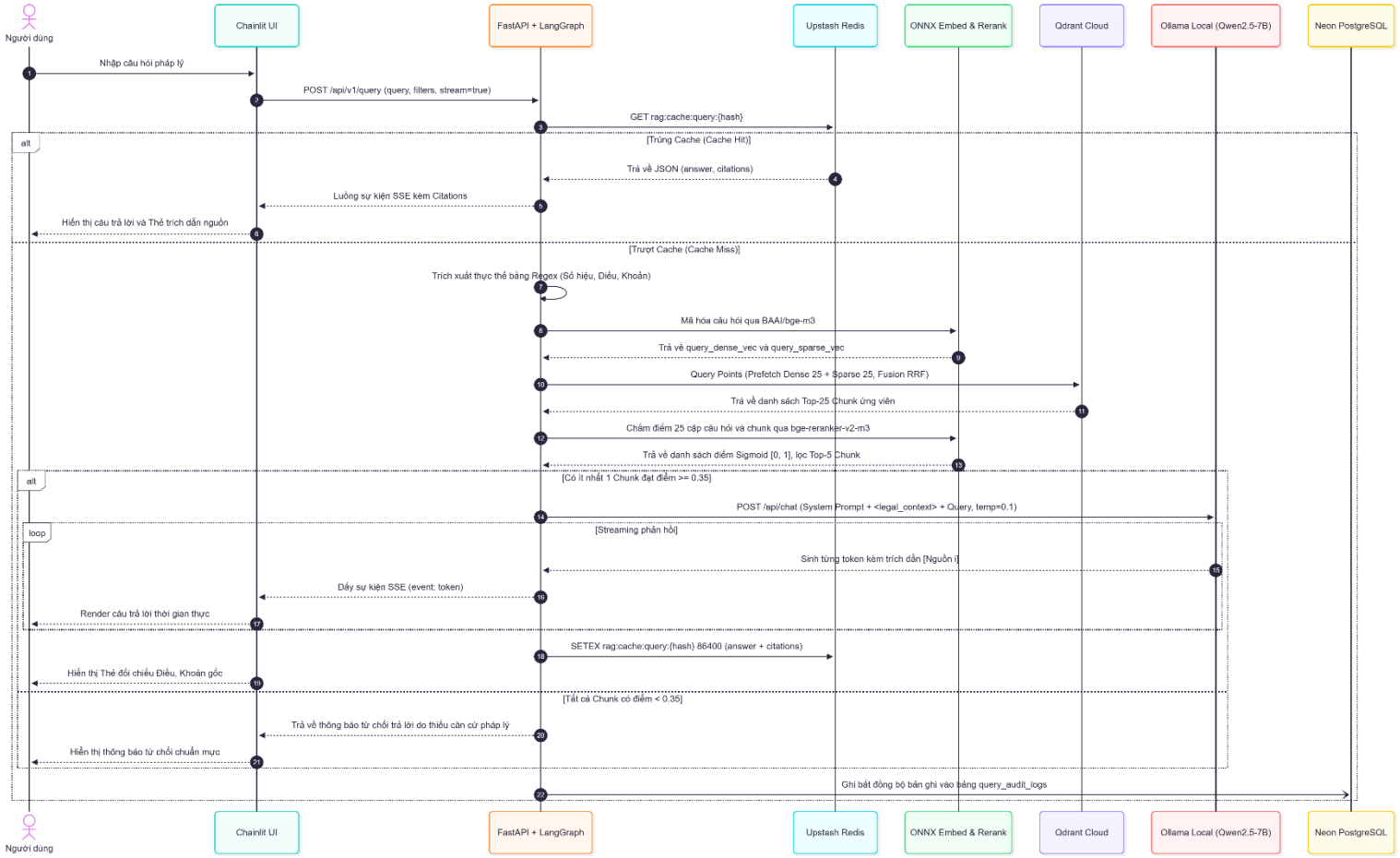
```
class LegalRAGState(TypedDict):
    request_id: str
    query: str
    filters: Dict[str, Any]
    cache_hit: bool
```

extracted_entities: Dict[str, str]
top_chunks: List[Dict[str, Any]]
abstained: bool
answer: str
citations: List[Dict[str, Any]]
latency_ms: Dict[str, int]

3.4.2. LangGraph Orchestration Activity Diagram

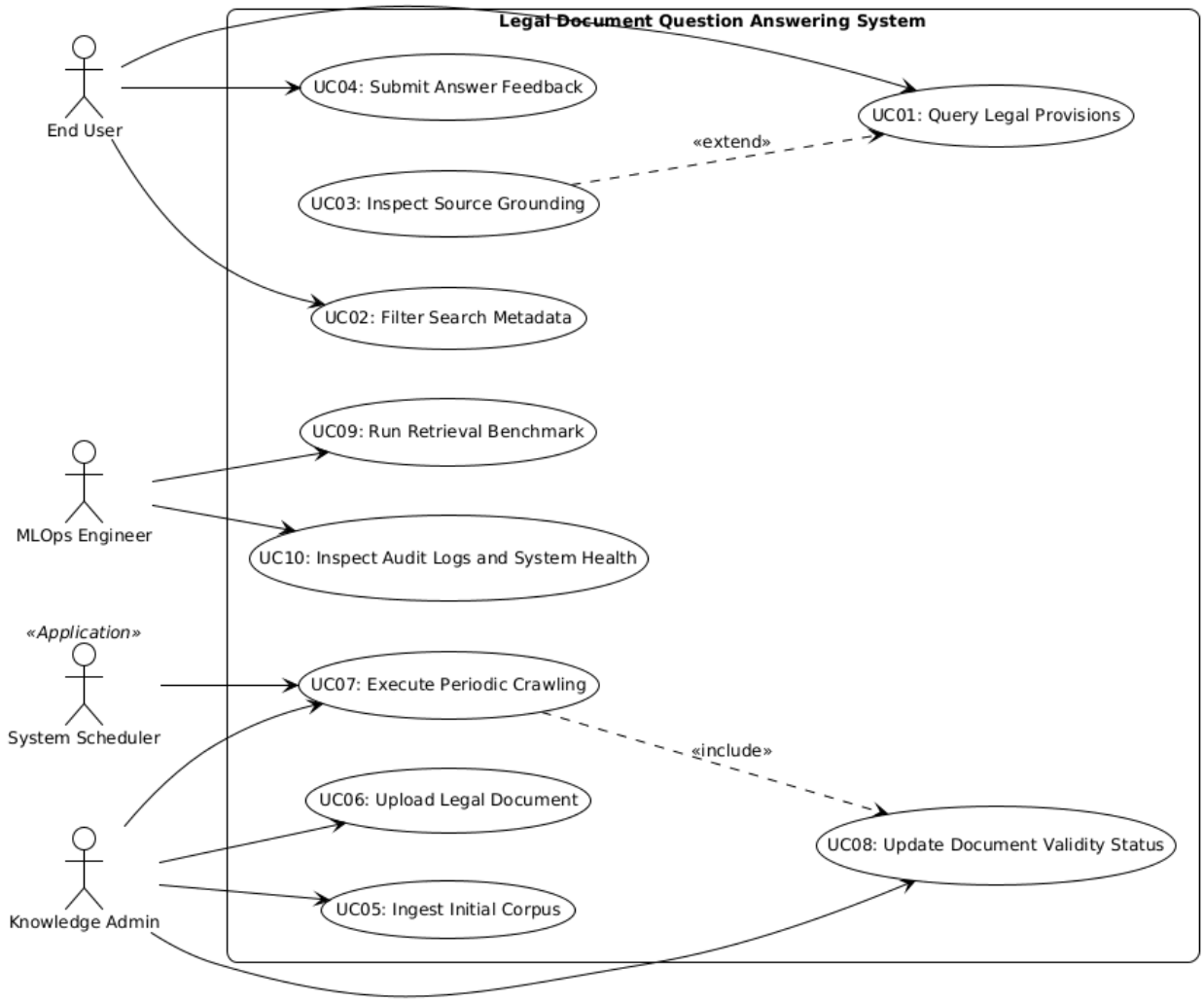


3.4.3. Biểu đồ Tuần tự Luồng Thực thi Truy vấn



CHƯƠNG 4. THIẾT KẾ GIAO DIỆN TÍCH HỢP VÀ API

4.1. Biểu đồ Use Case Tổng thể



4.2. Đặc tả Các Điểm cuối RESTful API

Phương thức	Đường dẫn Endpoint	Mô tả Chức năng	M
POST	`/api/v1/query`	Thực thi truy vấn hỏi đáp pháp luật, trả về JSON hoặc luồng SSE.	2
POST	`/api/v1/ingest`	Tải lên tệp văn bản `.docx/.pdf`, bóc tách cấu	2

Phương thức	Đường dẫn Endpoint	Mô tả Chức năng	M
		trúc và nạp lên Qdrant và PostgreSQL.	
PATCH	`/api/v1/documents/{doc_id}/status`	Cập nhật trạng thái hiệu lực của văn bản trên PostgreSQL và Qdrant Payload.	2
POST	`/api/v1/feedback`	Ghi nhận đánh giá của người dùng cho một `request_id` cụ thể.	2
GET	`/api/v1/health`	Kiểm tra kết nối tới Ollama Local, Qdrant Cloud, Neon PostgreSQL và Redis.	2

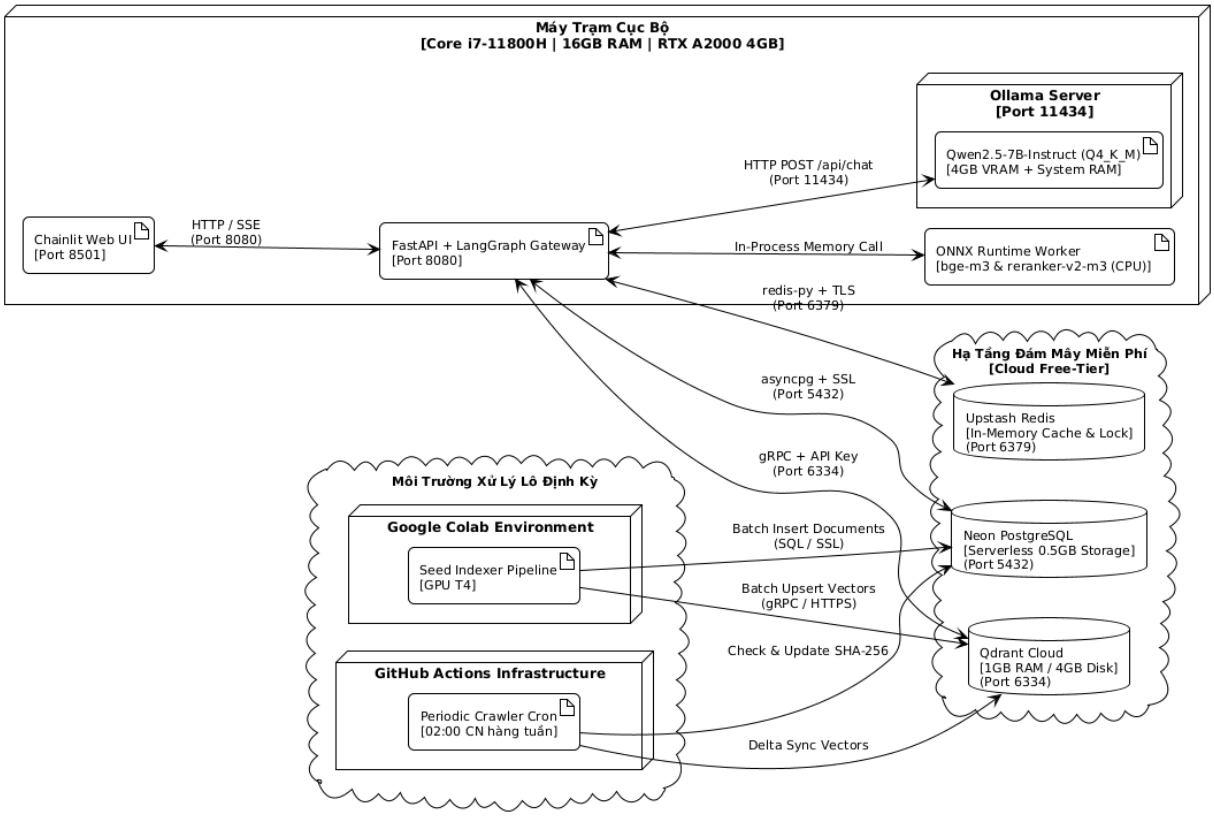
4.3. Giao thức Truyền tải Luồng Sự kiện SSE (`POST /api/v1/query` với `stream=true`)

Khi tham số `stream=true`, máy chủ trả về tiêu đề `Content-Type: text/event-stream` với 3 loại sự kiện tuần tự:

1. Sự kiện `metadata` (Gửi ngay sau khi hoàn tất bước Rerank):
   ```
   event: metadata
   data: {"request_id": "req_9021a8f4", "abstained": false,
   "citations": [...]}
   ```
2. Sự kiện `token` (Lặp lại cho mỗi token sinh ra từ Ollama):
   ```
   event: token
   data: {"delta": "Phạt tiền từ 400.000 đồng"}
   ```
3. Sự kiện `done` (Kết thúc luồng và trả thông số độ trễ):
   ```
   event: done
   data: {"metrics": {"retrieval_latency_ms": 110,
   "rerank_latency_ms": 280, "generation_latency_ms": 3200,
   "total_latency_ms": 3590}}
   ```

CHƯƠNG 5. THIẾT KẾ TRIÊN KHAI VÀ QUY TRÌNH THỰC HIỆN

5.1. Sơ đồ Triển khai Vật lý và Kết nối Mạng



5.2. Cấu trúc Thư mục Mã nguồn Dự án

```
legal-rag-system/
├── .env.example
├── requirements.txt
├── app/
│   ├── main.py              # Khởi tạo FastAPI và các route
│   ├── config.py            # Đọc và kiểm tra biến môi trường qua Pydantic Settings
│   ├── schemas/
│   │   ├── api_models.py        # Định nghĩa Request / Response JSON contracts
│   │   └── chunk_models.py      # Định nghĩa LegalChunkRecord
│   ├── services/
```

```
|   |   ├── parser_service.py      # Máy trạng thái cắt Chương, Điều, Khoản và Breadcrumb
|   |   ├── embedding_service.py   # Mã hóa BAAI/bge-m3 và chấm điểm Reranker ONNX
|   |   ├── retriever_service.py   # Giao tiếp Qdrant Hybrid Search RRF
|   |   ├── llm_service.py         # Giao tiếp Ollama cổng 11434 và System Prompt
|   |   ├── cache_service.py       # Giao tiếp Upstash Redis
|   |   └── db_service.py          # Giao tiếp Neon PostgreSQL và ghi Audit Log
|   └── workflow/
|       └── langgraph_rag.py       # Định nghĩa LegalRAGState, các Node và Conditional Edge
├── scripts/
|   ├── init_databases.py          # Tạo bảng PostgreSQL và tạo Collection Qdrant
|   ├── colab_seed_indexer.ipynb   # Notebook chạy trên Google Colab để nạp Seed Dataset
|   ├── periodic_crawler.py        # Script cào văn bản mới và cập nhật hiệu lực
|   └── evaluate_mlflow.py         # Script chạy 50 câu hỏi chuẩn và ghi log vào MLflow
└── ui/
    └── chainlit_app.py            # Giao diện hội thoại và thẻ đối chiếu trích dẫn
```

5.3. Đặc tả Biến Môi trường Hệ thống (`.env`)

1. Cấu hình Cơ sở dữ liệu Vector Qdrant Cloud

```
QDRANT_HOST="<qdrant-cluster-hostname>"

QDRANT_PORT=6333

QDRANT_API_KEY="<qdrant-secret-api-key>"

QDRANT_COLLECTION="legal_chunks_v1"
```

2. Cấu hình Cơ sở dữ liệu Quan hệ Neon PostgreSQL

```
DATABASE_DSN="postgresql+asyncpg://<user>:<password>@<neon-hostname>/legal_db?sslmode=require"
```

3. Cấu hình Bộ đệm Upstash Redis

```
REDIS_DSN="rediss://default:<upstash-password>@<upstash-hostname>:6379"

CACHE_TTL_SECONDS=86400
```

4. Cấu hình Máy chủ Suy luận Cục bộ Ollama

OLLAMA_HOST="127.0.0.1:11434"

OLLAMA_MODEL_NAME="qwen2.5:7b-instruct-q4_K_M"

LLM_TEMPERATURE=0.1

LLM_TOP_P=0.90

LLM_MAX_TOKENS=1024

5. Cấu hình Mô hình Truy xuất và Ngưỡng Kiểm soát

EMBEDDING_MODEL_PATH="BAAI/bge-m3"

RERANKER_MODEL_PATH="BAAI/bge-reranker-v2-m3"

TOP_K_RETRIEVAL=25

TOP_N_RERANK=5

RERANK_SCORE_THRESHOLD=0.35

5.4. Quy trình Triển khai Kỹ thuật Từng bước

- **Bước 1 — Khởi tạo Đám mây và Lược đồ:** Đăng ký cụm miễn phí trên Qdrant Cloud, Neon PostgreSQL và Upstash Redis; điền thông số vào tệp `.env` và chạy `python scripts/init_databases.py` để tạo bảng PostgreSQL cùng Collection `legal_chunks_v1` có bật nén `INT8 Scalar Quantization`.
- **Bước 2 — Nạp Kho Dữ liệu Nền trên Google Colab:** Mở tệp `scripts/colab_seed_indexer.ipynb` trên Google Colab (chọn GPU T4), đọc luồng dữ liệu từ `vohuutridung/vietnamese-legal-documents`, lọc các văn bản quy phạm còn hiệu lực, chạy `parser_service.py` để cắt theo `Điều/Khoản` kèm `Breadcrumb`, mã hóa bằng `BAAI/bge-m3` và đẩy trực tiếp lên Qdrant Cloud cùng Neon PostgreSQL.

- **Bước 3 — Cài đặt Mô hình Cục bộ:** Cài đặt Ollama trên máy trạm, thực thi lệnh `ollama pull qwen2.5:7b-instruct-q4_K_M` và tải trọng số ONNX của `bge-m3` và `bge-reranker-v2-m3` về thư mục cục bộ.
- **Bước 4 — Kiểm định Tầng Truy xuất với MLflow:** Chạy `python scripts/evaluate_mlflow.py` trên bộ 50 câu hỏi kiểm thử chuẩn để xác nhận `Recall@25 >= 92%` và `Hit Rate@5 >= 85%`.
- **Bước 5 — Khởi chạy Backend và Giao diện:** Chạy máy chủ FastAPI bằng lệnh `uvicorn app.main:app --port 8080` và khởi động giao diện người dùng bằng lệnh `chainlit run ui/chainlit_app.py --port 8501`.
- **Bước 6 — Nghiệm thu Toàn trình:** Thực thi kiểm thử 5 bộ kịch bản từ `SUITE-A` đến `SUITE-E` theo đúng tiêu chí của tài liệu SRS.